A Major Project Synopsis on

"SCRIPTSENSE: AUTOMATED MULTI-MODAL DOCUMENT TO TEXT EXTRACTION SYSTEM"

"

Submitted to Manipal University Jaipur

Towards the partial fulfilment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In Information Technology

2025-2026

By **Gaurav Singh Shekhawat**

229302573

MANIPAL UNIVERSITY JAIPUR

(University under Section 2(f) of the UGC Act)

Under the guidance of

Mr. Ravinder Kumar

Department of Information Technology

School of Computer Science and Engineering

Manipal University Jaipur

Jaipur, Rajasthan

1. Introduction

The modern world operates on an overwhelming volume of both physical and digital documents. Despite global advancements in digitization, a critical operational gap remains in reliably extracting unstructured visual data—specifically unconstrained human handwriting, heavily degraded printed documents, and standalone image artifacts—into machine-readable digital formats.

ScriptSense is a multi-modal document intelligence ecosystem that utilizes advanced concepts from Computer Vision and Deep Learning (Sequence-to-Sequence modeling) to recognize the context of highly varied document images and transcribe them into clean, structured digital formats like plain text.

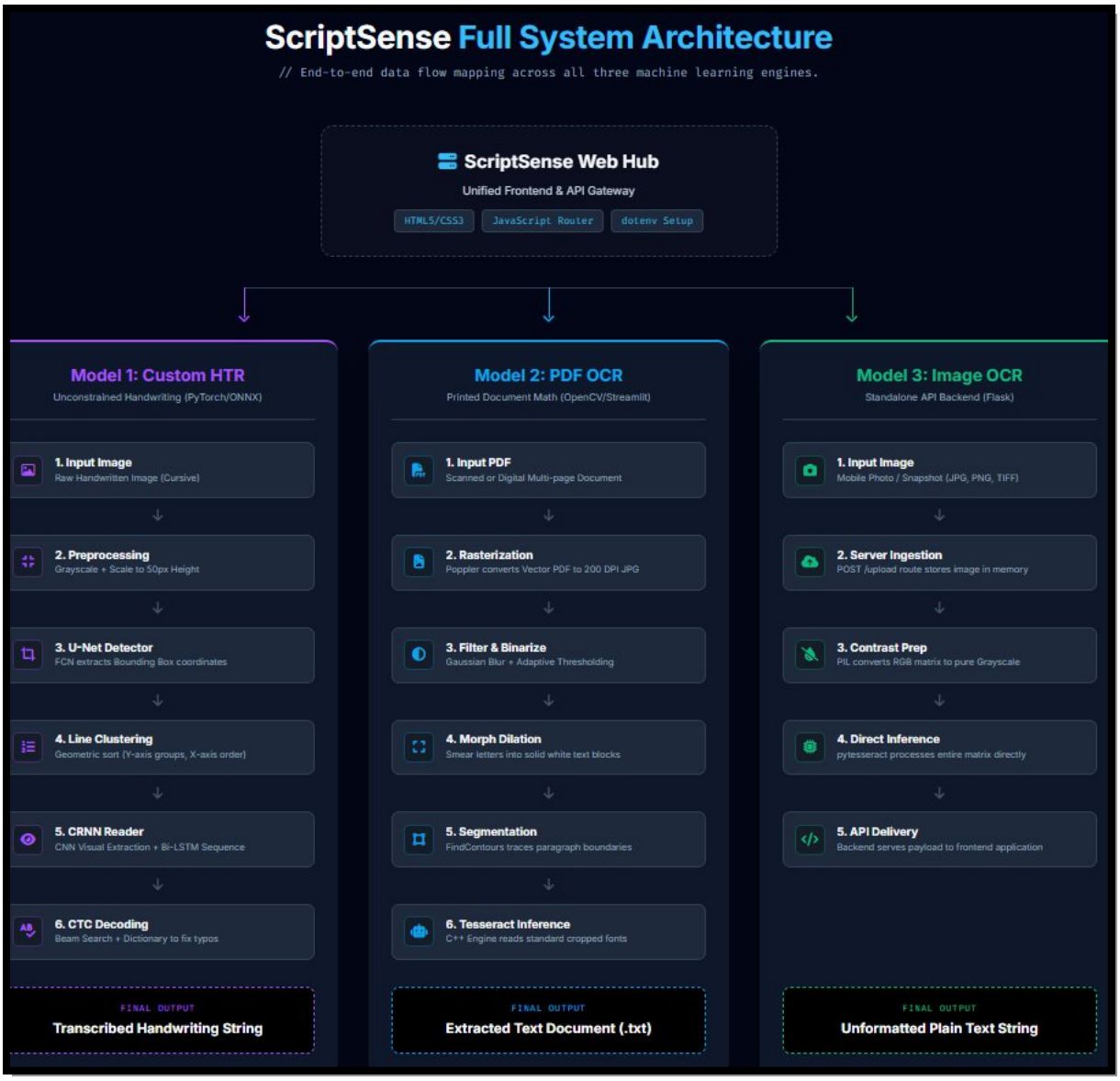

Figure 1. ScriptSense Full system architecture.

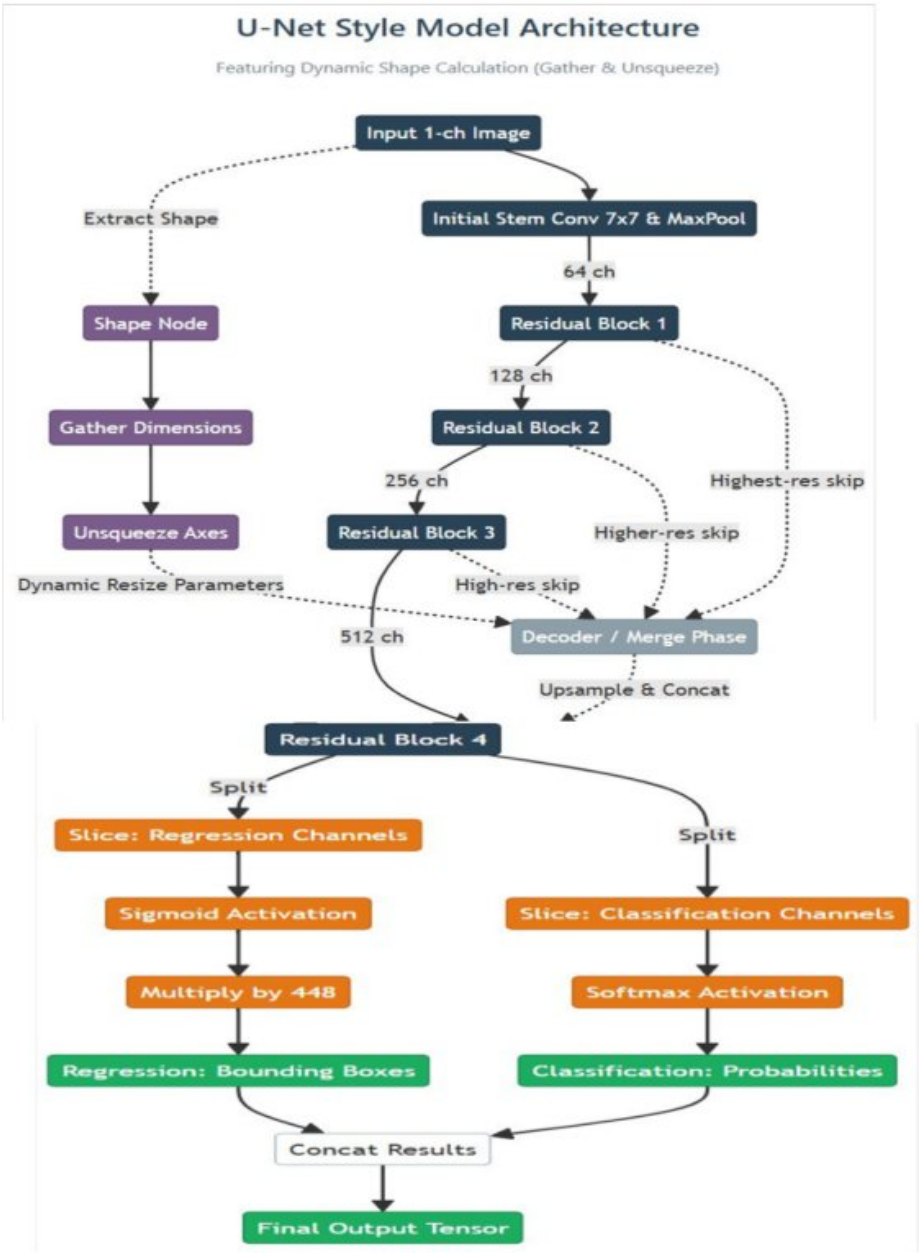

Figure 2. U-Net architecture for bounding boxes detection in Handwritten Text Recognition pipeline.

2. Motivation

Standard Optical Character Recognition (OCR) relies heavily on deterministic pattern matching and matrix comparisons. While successful for standard, clean digital fonts, this approach is highly brittle and completely falls apart when confronted with visual entropy. When humans write, they connect characters unpredictably, slant their words, and overlap strokes. Similarly, physical scanners introduce noise, lighting gradients, and shadows that render traditional OCR useless.

Furthermore, manual data entry is slow, highly expensive, and profoundly prone to human error. The motivation behind developing ScriptSense stems from these exact practical necessities:

1. **Healthcare (Aid to medicine):** Digitizing handwritten doctor prescriptions to prevent fatal medication errors caused by illegible handwriting.
2. **Finance & Legal:** Automating the bulk extraction of multi-page digital PDF documents and indexing massive scanned court records.
3. **Retail & Logistics:** Scanning physical store receipts and shipping labels directly from mobile camera photos to extract raw text streams rapidly, bypassing manual typing.

3. Project objective

The primary objective of our project is to learn the concepts of Convolutional Neural Networks (CNN), Recurrent Neural Networks (Bi-LSTM), and deterministic Computer Vision math to build a working, production-ready ecosystem of document extraction.
Specifically, the objectives are:

1. **Custom HTR Pipeline:** Implement an end-to-end handwriting recognition system using a U-Net detector and a CRNN reader optimized via ONNX runtime.

2. **CTC Dictionary Decoding:** Implement Word Beam Search to structurally eliminate spelling errors and hallucinations in handwriting predictions.

3. **PDF OCR Pipeline:** Build a high-speed matrix math pipeline using OpenCV (Adaptive Thresholding and Morphological Dilation) to clean degraded documents before passing them to Tesseract.

4. **Image-to-Text OCR Pipeline:** Develop a lightweight Flask-based ingestion system using Pillow (PIL) and Tesseract for rapid, direct text extraction from unstructured standalone photos (JPG/PNG).

5. **Unified Dashboard:** Deploy these systems via a highly responsive, dark-themed web hub utilizing Streamlit, Gradio, and Flask APIs.

4. Resources

ScriptSense does not rely on a single monolithic algorithm, but rather a specialized three-tier methodology:

(1) Model 1: Handwritten Text Recognition (HTR)

1. **Detection:** The input image is converted to grayscale and scaled dynamically. A Fully Convolutional Network (U-Net architecture) processes the image, outputting geometric bounding box coordinates for detected ink structures.
2. **Clustering:** A custom geometric line clustering algorithm sorts the randomly detected spatial boxes into a human-readable sequence (top-to-bottom, left-to-right).
3. **Reading:** A Convolutional Recurrent Neural Network (CRNN) processes the isolated word crops. CNN layers extract visual stroke features, while Bi-LSTM layers model the sequential context of the letters.
4. **Decoding:** Connectionist Temporal Classification (CTC) paired with a Word Beam Search Prefix Tree translates the raw probabilities into final words, aggressively penalizing non-dictionary spelling mistakes.

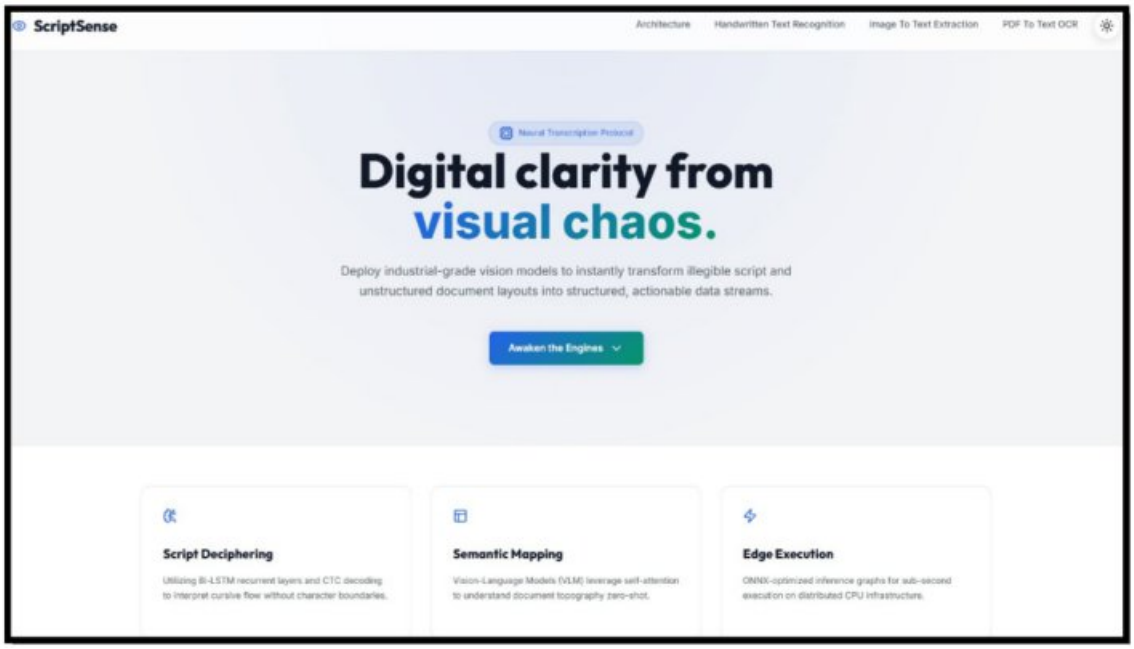

Figure 3. ScriptSense web dashboard interface.

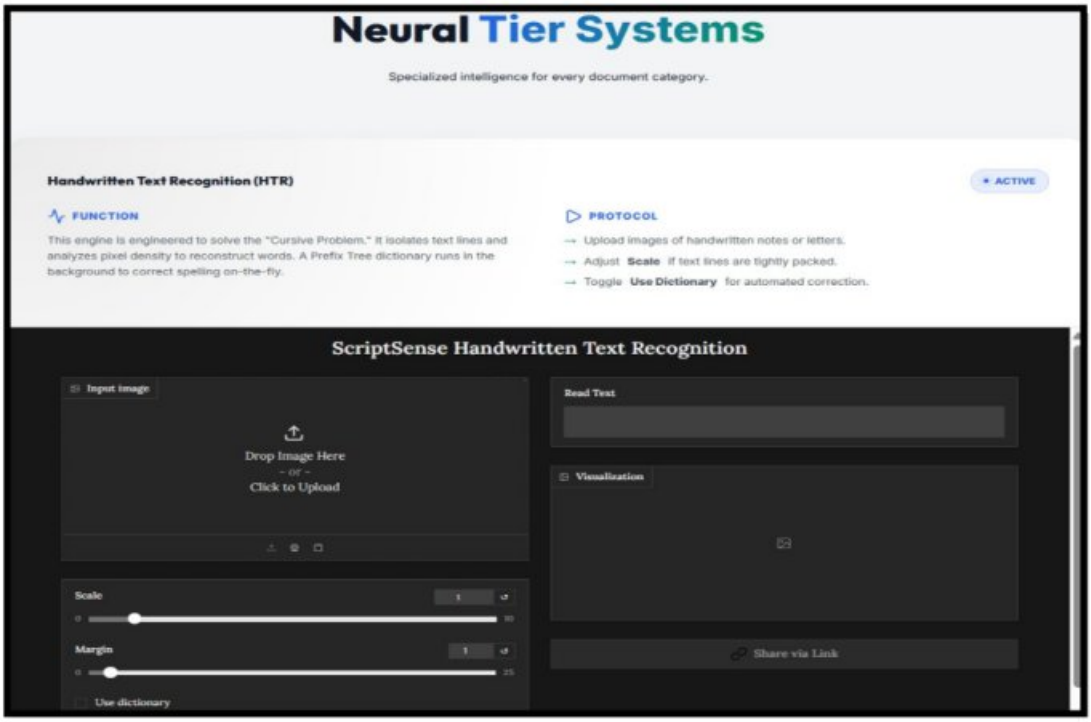

Figure 4. Model 1 HTR user interface.

(2) Model 2: PDF-to-Text OCR

1. **Sanitization:** Vector PDFs are rasterized into 200 DPI images. OpenCV applies a Gaussian Blur (low-pass filter) and Adaptive Binarization to calculate local lighting gradients, completely eradicating scanner noise and shadows.
2. **Extraction:** Morphological Dilation is applied with a specific rectangular kernel to intentionally smear letters together into solid paragraph blocks. Contours are drawn around these blocks, which are then passed to the Tesseract engine for perfectly clean font extraction.

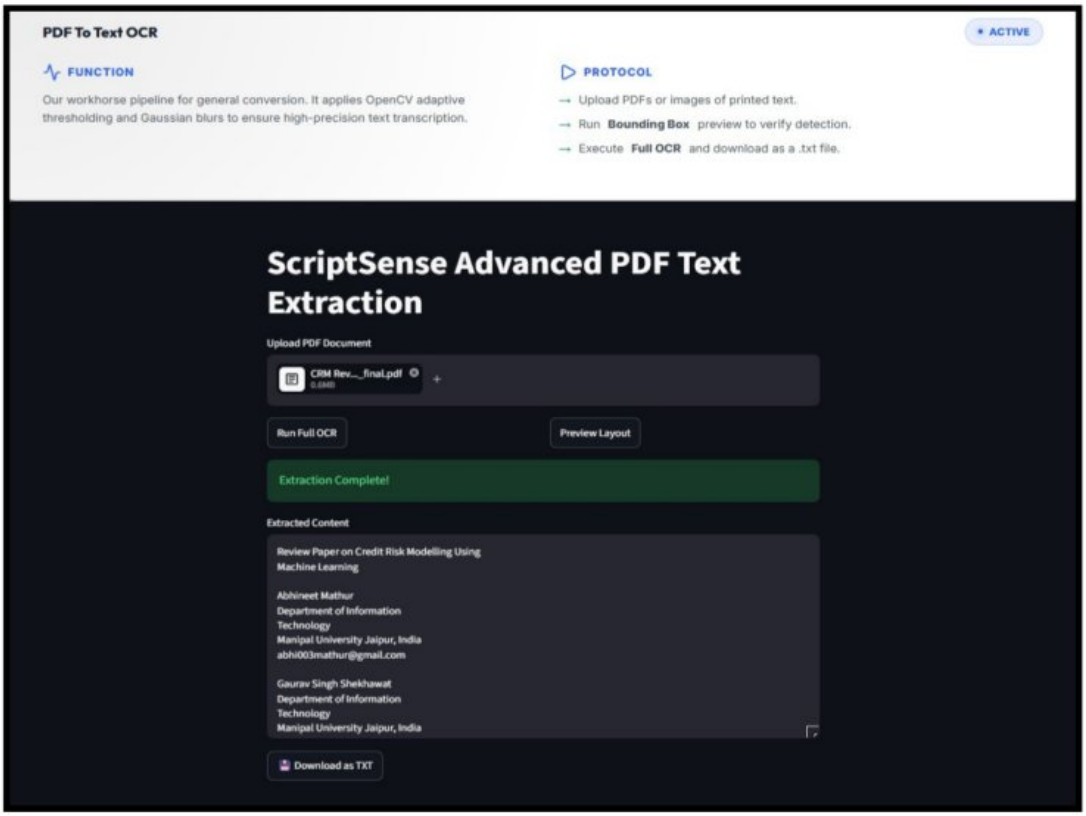

Figure 5. Model 2 PDF to text advanced extraction user interface.

(3) Model 3: Image-to-Text OCR Pipeline

1. **Ingestion & Preprocessing:** A Flask API securely ingests raw image files (JPG/PNG/TIFF). The Python Imaging Library (PIL) is then utilized to strip the color channels, converting the image to pure grayscale to maximize ink-to-background contrast.
2. **Inference & Extraction:** The high-contrast matrix is processed directly by the Tesseract OCR engine. Built-in LSTM networks map spatial geometries to standard character sequences, outputting a continuous, unformatted raw text string that preserves spatial line breaks.

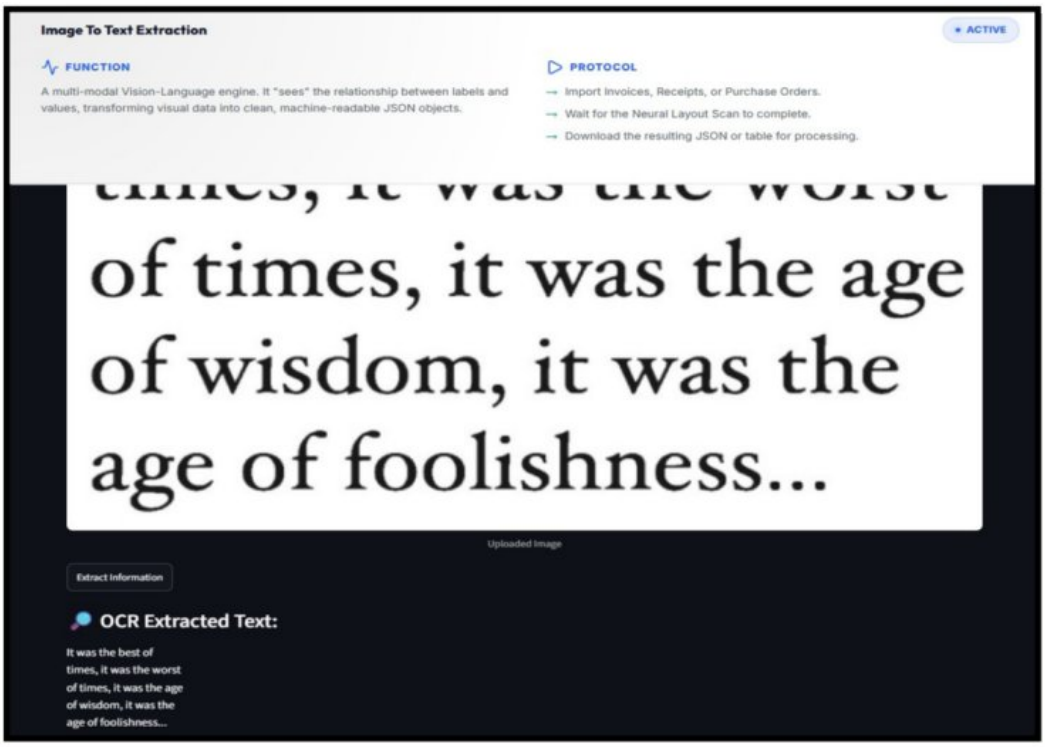

Figure 6. Image to text extraction OCR model user interface.

5. Resources

The resources used in this project are modern, production-grade frameworks designed for edge inference and cloud deployment. To implement the project, we will need a system with these specifications:

- A Windows/Linux based operating system (with WSL2 for deployment testing).
- Python 3.11.
- **IDE:** Visual Studio Code.
- **Deep Learning:** PyTorch (for training) and ONNX Runtime (for sub-second edge CPU inference).
- **Computer Vision:** OpenCV-Python (cv2), Pillow (PIL).
- **OCR Engines:** Tesseract-OCR (v5.0+), pdf2image, poppler-utils.
- **Web Frameworks:** Flask, Streamlit, Gradio.
- **Datasets:** IAM Handwriting Database 3.0 (for HTR neural weights).

6. Bibliography

The following resources and literature will be used to gain knowledge about the topic:

1. O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *MICCAI*, Springer, 2015.
2. B. Shi, X. Bai, and C. Yao, "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition," *IEEE TPAMI*, 2017.
3. A. Graves, et al., "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," *ICML*, 2006.
4. H. Scheidl, "Word Beam Search: A Read-Out Algorithm for Sequence-to-Sequence Models," *arXiv preprint arXiv:1809.00619*, 2018.
5. R. Smith, "An Overview of the Tesseract OCR Engine," *ICDAR*, 2007.
6. ONNX Runtime Documentation: https://onnxruntime.ai/docs/
7. OpenCV Morphological Transformations: https://docs.opencv.org/

7. Timeline